

Project Summary/Abstract

Computational models of complex systems are playing an increasingly important role in the social and life sciences. Unfortunately, most efforts along these lines depend on one modeling paradigm to the exclusion of others. We maintain that scientists would be better served by frameworks that incorporate ideas from these different modeling traditions and thus combine their separate strengths.

We propose to develop an innovative modeling environment that addresses this need. The software will incorporate a unified formalism that combines aspects of agent-based models, differential equations, rule-based systems, and social networks. Users will access the environment through a standard Web browser, thus avoiding installation on local machines and enabling long-distance sharing of models. A graphical interface will simplify model entry and support visualization of model structures. Simulation will take place on a back-end server while the resulting trajectories will be displayed in the Web browser. The environment will also incorporate novel methods for searching the space of model structures, and their associated parameters, for candidates that explain available observations.

We intend to demonstrate the new framework on a variety of modeling tasks that involve interactions among social, health, and environmental variables, and we plan to evaluate our algorithms on their ability to handle both natural and synthetic data sets. We also aim to build a user community that will provide feedback to let us improve the software environment. Our technical team has the expertise in computational modeling, interface design, and model discovery needed to achieve its vision, and the four-year time line will let us design, implement, evaluate, and refine a robust, accessible, and usable environment that provides flexible support for modeling complex systems.

Significance

We propose to develop a unified modeling formalism, along with an associated Web-based modeling environment, that has the potential to transform the construction of formal models in the social and life sciences. Previous modeling efforts have been hampered by their reliance on one or two modeling paradigms, whereas our unified approach will let scientists draw on powerful ideas about agent-based models, differential equations, rule-based systems, and social networks. The modularity of our framework will ease the creation and revision of models, and the ability to retrieve, edit, and simulate these models within a Web browser will make the modeling enterprise accessible to a much broader community. Our approach should also make sharing of models, and their joint development, much easier than in the past. Finally, our computational methods for search through the model space could revolutionize the modeling process by reducing the effort of human scientists and by making model construction, comparison, and evaluation far more systematic.

The framework that we have proposed and the software environment that we will develop are quite general. They should be relevant to any branch of the social or life sciences that involve dynamic phenomena, but they hold special promise for settings that involve groups of humans that engage in rule-guided social interaction over time and space. Our approach holds broad potential for explaining interactions among social, environmental, and health factors, and it should prove especially useful for situations which involve limited amounts of data, and which thus benefit from models that incorporate knowledge-based hypotheses about the agents, processes, and relations in operation. Our modeling environment should provide researchers with substantial assistance in their efforts to understand the behavior of complex human systems.

Innovation

The social and life sciences have seen increases in the size and complexity of both the data they collect and the models they develop to explain them. Computational methods offer a natural response to this situation, but previous efforts on modeling and simulation have emphasized one framework at the expense of others. In contrast, we propose to develop a new type of interactive modeling environment that will introduce three important innovations:

- Our environment will offer a unified formalism that incorporates ideas from, and that combines strengths of, four traditionally distinct paradigms: agent-based models, differential equations, rule-based systems, and social networks.
- Our software will incorporate a user interface that operates within a Web browser, which in turn will simplify model entry, support visualization, and enable model sharing, along with a back end that handles model storage and simulation on remote servers.
- Our framework will introduce novel computational methods for automating search through the space of model structures and their associated parameters, guided by their fits to time-series data, and thus aid scientists in constructing explanatory models.

Each of these contributions distinguishes our approach from earlier research on modeling complex systems, but the most important innovation lies in their combination into an integrated system that supports the specification, visualization, simulation, sharing, and discovery of scientific models that draw on different explanatory metaphors. No other efforts in modeling and simulation have attempted to support these facets simultaneously, making the proposed system the first of its kind.

Approach

Challenges in Computational Modeling

There is growing agreement that formal models have an important role to play in the biomedical and social sciences, as they have long done in other scientific disciplines. However, the models required in these fields must often be quite complex, especially when social factors come into play. Recent years have seen substantial progress in modeling frameworks, but these advances have themselves raised new challenges that demand additional research. Here we review three such issues that we believe require attention, in preparation for discussion of our proposed responses later in the proposal.

The first challenge involves the diversity of paradigms that have emerged for modeling complex systems, especially those within the social and life sciences. Differential equation models, often associated with the system dynamics movement (Forrester, 1961), characterize dynamic behavior as a set of linked equations, typically at an aggregate level. Rule-based systems, in particular production systems (e.g., Klahr, Langley, & Neches, 1987), were originally developed to model individual human cognition and explain behavior in terms of conditional responses, typically over symbolic structures. Agent-based models (e.g., Epstein & Axtell, 1996), used in both the social and environmental sciences, instead characterize systems as a set of agents that interact over time. Finally, social network accounts (e.g., Borgatti et al., 2009) focus on structural relations among people, typically without explaining system dynamics. Although each paradigm has important ideas and methods to offer, they also have limitations, and the social and life sciences would be better served by a framework that combines their individual strengths. We propose to develop such a unified modeling formalism in our research.

The second challenge revolves around the process of constructing and utilizing models of biomedical and social phenomena. There is broad consensus that scientists can benefit from computational aids that let them specify, store, retrieve, and simulate these models. Such tools should be easily accessible (e.g., they should not require downloading and installation) and simple to use, letting users translate their theoretical ideas into well-defined models even when they have little or no training in computer programming. These computational aids should let scientists create and update models, visualize and inspect their content, generate predictions and compare them to observations, measure model fits to data, and share results with other researchers. We propose to develop an interactive software environment that has these characteristics.

The third challenge follows from the view that science depends on heuristic search through a space of candidate models (Bradshaw, Langley, & Simon, 1983). Although scientists are often adept at carrying out this search, it is also clear that they can benefit from computational aids that consider more fully and systematically the space of models. Such mechanisms can assist not only in estimating model parameters to improve fits to observations; they can also search through a space of model structures. Moreover, this search can be guided not only by the data the models aim to match, but also by background knowledge about the domain. Using such knowledge can not only increase the chances that scientists will find the generated structures more plausible; it also supports the construction of models that go beyond description to provide explanations of data. We propose to develop computational methods to carry out search for explanatory models stated in our unified formalism.

Although recent years have seen development of a variety of modeling environments that are now widely used, we believe that our approach is novel in its response to these three challenges. For example, the STELLA environment (Richmond, Peterson, & Vescuso, 1987), which supports models organized in the system dynamics tradition (Forrester, 1961), has a long and successful history within the environmental and social sciences. However, STELLA and its relatives typically model behavior only at the aggregate level, and they emphasize differential equations to the exclusion of other modeling paradigms. Rule-based frameworks like Soar (Laird, 2012) and ACT-R (Anderson & Lebiere, 1998) have been widely adopted for modeling individual human cognition, but they emphasize behavior on symbolic tasks and they are not designed for explaining interactions among large groups.

Agent-based frameworks like NetLogo (Wilensky & Rand, in press), Repast (North et al., 2006), and MASON (Luke et al., 2005) support models that involve many interacting agents, but they emphasize this idea over other metaphors. Although they let users embed conditional statements and difference equations within their agent programs, these ideas enter as second-class citizens.¹ The AnyLogic platform supports system dynamics, agent-based, and discrete-event models, but these appear to involve three separate modules with no facility for integrating them. Finally, traditional models of social networks have focused on static relationships, rather than attempting to explain how such connections change over time. To our knowledge, no extant modeling environment offers a unified formalism that combines ideas from the different paradigms in a balanced manner. We believe our approach offers substantial benefits to scientists, who will be able to draw on the strengths of multiple metaphors when constructing their models.

We also maintain that our environment will prove easier to use than existing software, especially for users who have limited modeling experience. One advantage lies in the modularity of our formalism, which lets users decompose models into simple chunks. This has considerable advantages over the procedural syntax adopted by agent-based frameworks, which derive from languages like Logo and Java, although our framework shares this feature with system dynamics and rule-based models. The Web-based character of our modeling environment also offers benefits in accessibility and reduced start-up costs when compared to environments that require users to download and install software before they begin modeling. Finally, our approach is further distinguished by its support for systematic search of model spaces. Some environments like MATLAB include methods for parameter estimation, but there has been very little work on methods that explore the space of model structures.

In the next section, we present an example scenario in which such a unified modeling framework would prove useful. After this, we describe our plans for constructing a modeling environment that addresses the three challenges outlined above. We also report preliminary results with an initial version of this environment. We then describe our research schedule, testing plans, and project personnel, after which we close by summarizing the main features of our proposal.

A Motivating Modeling Scenario

Consider an increasingly common situation that involves interactions between health and social factors. Suppose an international traveler has brought a new strain of an infectious disease into the country, but its unfamiliarity means that the details of its operation are poorly understood. Relevant characteristics might include the length of the infection period, the probability of transmission, and the vectors that pass it on. Further suppose that some preliminary data are available, but that these are incomplete and a better understanding of the disease is needed to form policy recommendations.

A formally specified, computational model of this disease's behavior would very useful for this purpose, but a complete account might need to combine ideas from different modeling paradigms.² For instance, differential equations can mimic the growth of infectious disease in individuals and the associated immune response, but its transmission might be better explained in terms of interactions among independent agents. Moreover, both intra-agent and inter-agent changes may be conditioned on environmental factors like the weather and on agent beliefs, say about symptoms that reveal when others are infected. Finally, relationships among agents may also come into play, leading the contagion to spread more rapidly in some groups.

In such situations, decision makers would benefit greatly from having access to reasonably accurate models soon after the disease onset. This would depend on being able to generate plausible models rapidly, simulate their behavior, and compare their predictions to the limited data available. Modelers should also be able to modify a model to reflect alternative health policies and simulate the variants to determine which would be most effective. Moreover, the set of candidate models is large enough that they would benefit from computational aids which search the model space systematically to find accounts that match current obser-

1. However, Lotzmann and Meyer (2011) report a rule-based extension to Repast that begins to address the first issue.

2. Lee et al. (2008) report some progress in this direction using an agent-based model of influenza that incorporates ideas about social networks.

Table 1. Specification for some entity or agent types in a model of infectious disease.

entity Person	entity Uninfected
x = [0, 100]	is-a Person
y = [0, 100]	disease_cells [0, 0]
id = (unique)	
immune_cells = [1, 1]	entity Infected
immune_rate = [0.1, 0.9]	is-a Person
immune_threshold = [200, 800]	disease_cells [10, 20]
immune_effectiveness = [0.2, 0.8]	
interaction_time = [5, 10]	

ventions. In addition, models should be easily shared with others in different locations, suggesting that they be accessible through the World Wide Web.

We maintain that a modeling environment with these capabilities would prove invaluable to both medical researchers and to policy makers. Of course, such a computational aid would not be limited to understanding infectious disease. It would be equally useful in modeling outbreaks of foodborne illness, the effects of pollution and other environmental factors on health, and even the effectiveness of different communications media in altering health-related behavior. The benefits of general, flexible, and usable software that supports a broad range of modeling formalisms in a single framework would be profound. In the next section, we describe an approach to achieving this vision.

A Unified Computational Modeling Environment

The main body of our research will involve designing, implementing, and testing a software environment that unifies ideas from the four modeling traditions that we have discussed. In this section, we describe our plans for this work, organizing them in terms of the three challenges stated earlier.

Designing A Unified Formalism

The most basic step in specifying a modeling framework involves selecting a representation for models. Since our goal is to develop a framework that unifies ideas from different modeling traditions, we need a formalism that incorporates those ideas. Moreover, it is important that it do so without giving primacy to one paradigm or another. We propose to develop a formalism that achieves this goal via four basic constructs.

AGENTS TYPES AND INSTANCES

First, the formalism will assume that a model includes a set of *entities* or *agents*, each with an associated type. Every agent type will have a set of associated attributes that take on either continuous or discrete values. Some attributes will serve as model parameters, in that they do not change over the course of a run, while others will serve as variables, in that they are dynamic. Every attribute will also include a probability distribution over its initial value. In addition, a model will indicate the initial number of agents for each type.

Table 1 shows an example agent type, corresponding to an individual *Person*, that might occur in a model of the infectious disease scenario described earlier. This includes attributes for the agent's *x* and *y* positions, as well as one for the number of immune cells it contains, all of which will change over time. Other attributes, such as the growth rate, effectiveness, and detection threshold of the immune system, will be constant and serve as model parameters. Each attribute also specifies the range of initial values that it may be assigned at the outset of a simulation run. The table also shows two specializations of *Person*, *Infected* and *Uninfected*, that denote agents that begin the simulation with and without infections, respectively. These agents inherit the attributes and ranges of *Person* but also introduce a new attribute, *disease_cells*, for which they have different distributions. Although this example focuses on individual humans, an agent may also denote an aggregated group, such as a household, or even the entire environment being modeled, such as a city.

Table 2. Some processes from a model of infectious disease, including conditions for activation.

```

process disease_growth(Person ?p1)
  virulence = 0.3
  d[?p1.disease_cells] = virulence * ?p1.disease_cells
process immune_response(Person ?p2)
  if ?p2.disease_cells > ?p2.immune_threshold
  then d[?p2.immune_cells] = ?p2.immune_rate * ?p2.immune_cells
process immune_effect(Person ?p3)
  if ?p3.disease_cells > 0
  then d[?p3.disease_cells] = -1 * ?p3.immune_effectiveness * ?p3.immune_cells
process move_toward(Person ?p4, Person ?p5)
  if ?p4.target = ?p5.id and distance{?p5.y, ?p4.y, ?p5.x, ?p4.x} > 1
  then d[?p4.x] = (?p5.x - ?p4.x) / distance{?p5.y, ?p4.y, ?p5.x, ?p4.x}
    d[?p4.y] = (?p5.y - ?p4.y) / distance{?p5.y, ?p4.y, ?p5.x, ?p4.x}
process transmit_disease(Person ?p6, Person ?p7)
  trans_percent = 0.01
  if ?p6.target = ?p7.id and distance{?p7.y, ?p6.y, ?p7.x, ?p6.x} < 1
  then d[?p6.interaction_time] = -1
    d[?p7.disease_cells] = trans_percent * ?p6.disease_cells

```

We must distinguish between agent types, like those in Table 1, and agent *instances*, which are necessary to enable model simulation. As noted, the former specify distributions over a type's attribute values. In contrast, the latter commit to particular values that are sampled from those distributions. Distinct agent instances have the same attributes, but they typically differ in their initial values. This distinction is needed because an operational model must commit to a particular set of agents for each type and to a specific set of initial attribute values. However, users need only worry about specifying agent types, along with the number of each type, as we clarify in the next subsection.

PROCESSES AND EQUATIONS

Agents provide the basic structural components of a model, but we also require some way to specify their dynamic behavior. Differential equations offer a notation for change in continuous attributes, but they do not let one encode the idea that some changes occur jointly.³ In response, our models will include a set of *processes* that comprise a set of equations that stand or fall together (Forbus, 1984). Each process will contain one or more algebraic equations (to describe instantaneous relations), differential equations (to describe changes over time), or discrete updates (to specify changes in discrete attributes).

The notion of a process combines ideas from equation-based modeling frameworks, which describe relations among variables, and rule-based systems, which support multiple effects. Table 2 presents five processes taken from a preliminary model for the infectious disease scenario. The first three processes, which describe the details of disease growth and immune response, each take only one argument of type *Person* and specify one differential equation. The dot notation lets us denote the attribute values of particular entities. For instance, *?p1.disease_cells* in *disease_growth* refers to the number of disease cells in whichever agent binds to the argument *?p1*. The equation in this process describes the change in this attribute but also refers to the disease *virulence*, an attribute associated with the process.

The final two processes in the table differ in that they each take two arguments of type *Person* and they each include two equations. The first describes the movement of one agent, *?p4*, toward another agent, *?p5*, with the former's coordinates being (*?p4.x, ?p4.y*) and the latter's being (*?p5.x, ?p5.y*). The second process includes an equation related to transmission of the disease from one agent to another. Another equation

3. Frameworks like Boolean networks (Kaufmann, 1993) and dynamic Bayesian networks (e.g., Page & Ong, 2006) can model changes in discrete attributes, but they cannot easily specify when such changes occur together.

decreases a counter that indicates how much time the first agent intends to continue interacting with the second, with longer interactions leading to increased transmission. These do not constitute the entire model, but they should clarify the process syntax.

Agents and processes play equal roles in our formalism. Without processes, agents are only static structures; without agents, processes have no intermediates to pass on their effects. However, we do assume that first argument serves as primary agent in each process, so that the collection of processes in which a given agent plays that role, taken together, serve to characterize its behavior. For example, the first agent in the *transmit_disease* process, which already has the disease, is the active party; the second, which does not, is passive and is influenced only as a side effect. Similarly, the first argument in *move_forward* is responsible for the change in position, while the second argument serves only as the target.

CONDITIONS ON PROCESSES

Although processes let us specify a set of effects that an agent can have on itself or other agents, we may not want these effects to take place in all situations. In response, our formalism incorporates a third idea that it borrows from rule-based models: conditional behavior.⁴ To this end, we can associate one or more conditions with each process that specify when it is active. These conditions are conjunctive, in that all must be satisfied for the equations to be applied. The attributes mentioned in these conditions must either be associated with the agents provided as arguments to the process or be internal to the process.

Some of the entries in Table 2 incorporate conditions of this sort. For example, the second process (*immune_response*) states that the number of the agent's immune cells only increase if the number of disease cells exceeds an agent-specific threshold. Similarly, the fourth process (*move_toward*) specifies that one agent approaches a second agent only if the latter has been selected as the target of the first and if their distance is greater than one. Conditions let our formalism encode models that involve discontinuous processes, which seems key to explaining some forms of nonlinearity that arise in complex systems.

SOCIAL RELATIONS

Our modeling formalism also incorporates a fourth idea, taken mainly from social network theories, that behavior is influenced by relations that hold among individuals. The notation for agents described above does not by itself let us encode such relations, since each attribute can take on only one value. Thus, we might use it to indicate that agent *Abe* has friend *Bob*, but not that *Abe* has other friends. To provide this representational power, we must be able specify that an attribute takes on a *set* of different values. When a process refers to such an attribute, as in the expression *if ?person1.friend = ?person2*, the variable *?person2* may refer to any member of this set.

However, we may also want to specify the strength, duration, or another numeric aspect of a relationship. To this end, we can extend the dot notation to associate one or more secondary attributes with the value of a primary attribute. For instance, suppose we want a process to refer to both the strength and the duration of person *?p1*'s friendship with another person, *?p2*. We can describe these secondary attributes with the expressions *?p1.friend:?p2.strength* and *?p1.friend:?p2.duration*, with the use of the pattern-match variable *?p2* in both cases ensuring that these denote the same friend. As with primary attributes, the values of such secondary attributes can appear in the left-hand sides, right-hand sides, and conditions of processes, letting models state how the characteristics of relationships influence each other.

Note that this notation encodes the view of the agent *A* to which the attributes are attached. The agents mentioned in these attributes may have quite different views of their relations to *A*. Thus, the extended formalism will let models encode asymmetrical connections, which are a key aspect of some social network theories. However, our framework moves beyond traditional accounts of social network effects, which do not typically emphasize change over time.

4. Of course, conditional statements also occur in other paradigms, including procedural languages, but they play a more central role in rule-based systems, where all behavior is conditional.

CONNECTIONS TO TRADITIONAL FRAMEWORKS

As we have indicated, our unified modeling formalism incorporates ideas from multiple traditions. The framework also subsumes these approaches as special cases. For instance, one can specify:

- agent-based models that include only discrete actions;
- differential equation models in which the environment is the single agent;
- rule-based systems that model the behavior of an isolated individual; and
- social networks that describe static algebraic relations among agents.

Thus, the new formalism should lose no representational power over older notations, while supporting the statement of hybrid models that would be either awkward or impossible in prior approaches.

We should also note that traditional notations offer different sources of modularity during model construction. Agent-based accounts provide modularity by combining the effects of many independent but interacting entities. In contrast, rule-based models are decomposed into a set of distinct but interacting condition-action elements. Traditional differential equations carve things up differently, organizing models into one equation for each variable or attribute. Finally, social network accounts treat inter-agent relations as the fundamental model element. Each type of modularity makes models easier to construct and analyze, and our formalism incorporates them all in a coherent way.

Creating, Utilizing, and Evaluating Models

Naturally, we want more than a unified formalism for stating behavioral models. We also want an interactive software environment that lets us create, utilize, and evaluate models specified in the formalism. We propose to develop an interactive environment that is both widely accessible and usable by scientists with little experience in formal modeling. We can divide this functionality into three separate but interrelated modules.

ACCESSING, VIEWING, AND EDITING MODELS

Before researchers can utilize models, they must first create them or, if they already exist, access them in some manner. To increase the accessibility of our modeling environment, we intend to make it available on the World Wide Web. Users will be able to log on to a Web site and select a model from among existing set, which they can then view and edit if desired. Alternatively, they can create a new model, to which they can add agents and processes, or copy an existing model, which they can then adapt. The Web-based interface means users will not have to install software on local computers, which should lower barriers to entry. Researchers will also be able to view each others' models and thus collaborate on their development.

The interface for model creation and editing will be highly structured so that users need not master a complex syntax. Basic editing operations will include adding an agent type, adding an attribute to an agent type, and specifying a range of values for a given attribute. The environment will also incorporate editing operations for adding a new process, specifying the types of agents that serve as its arguments, adding an attribute associated with the process (and its default value), introducing a condition on a process, and adding an algebraic, differential, or discrete equation. The software will ensure that each equation obeys the proper syntax, as well as checking that every attribute it mentions has been declared earlier. The environment will also let users revise elements if they decide to change model names or content after their initial creation.

Although we have focused on textual notations for models, the environment will also let users visualize their structure and content in graphical terms. They will be able to display the different types of entities and their associated attributes. On request, the system will also display how processes connect agents' attributes, as well as the social relations between agents.⁵ We will also adopt the general philosophy that every text-based operation for model editing should have a corresponding graphical operation. This will let users draw on the interaction mode that best matches their personal preferences.

5. In addition, we will explore ways to adapt these techniques to display changes in the values of agent attributes during model simulation, which we discuss shortly.

SIMULATING MODELS TO PREDICT BEHAVIOR

The central operation of our environment will involve simulating models to predict trajectories over time. In order to carry out such simulations for our unified notation, we must incorporate mechanisms for each of the traditions from which it borrows. As in the agent-based, difference-equation, and rule-based frameworks, the environment must transform the state of the model on the current time step into the state on the successive time step. Repeating this calculation for a specified number of steps produces a trajectory of model states that, taken together, constitute predictions about the values for all model variables over the desired period.

However, recall that the user specifies a model in terms of agent and process *types*, along with the desired number of agents. To make such models operational, the environment must generate a set of agent *instances* with attribute values sampled from the given distributions. The system must also generate a set of *grounded* processes that include such agent instances as arguments. The user-provided entities and processes serve as templates for these grounded, operational versions of the model that are needed for simulation. In many cases, multiple simulations runs will be needed over different groundings with distinct initial values for agents.

Because a grounded model will include many distinct agents, each step in the simulation process will involve iterating over them, as in agent-based frameworks. Because agent behavior is encoded in terms of processes, it will also require iterating over each process in which that agent plays the primary role, as discussed earlier. And because a process may contain multiple equations, each simulation step will also involve iterating over each equation, updating the values of attributes on their left-hand sides. However, this will occur only for active processes whose conditions are satisfied, which otherwise will have no effect.

Earlier we noted sources of modularity in our modeling formalism. These offer great advantages in model specification, but they also raise issues for simulation, since different agents and different processes can influence the same attributes. We will address this issue by associating, with each attribute of each agent type, a simple arithmetic function that states how to combine influences on that attribute. The default combining function will be summation, but the environment will also support multiplication, maximization, minimization, and averaging, all of which we have observed in our preliminary modeling efforts.

An important special case of such operations involves calculating aggregations over sets of agents. Although users may want to examine the trajectories of individual agents' attributes over time, more typically they will want to know the sums or averages of attributes for entire populations of given agent types. In our infectious disease scenario, we might want to calculate the number of infected people as a function of time, as well as the average degree of infection. The environment will calculate these aggregated values for each time step by summing or averaging over the specified attribute values on that step.

VISUALIZING PREDICTED AND OBSERVED TRAJECTORIES

Users will be able to invoke the simulation module by indicating the number of time steps and the step size, along with the desired number of runs. The environment will take these specifications, simulate the current model the specified number of times, and store the resulting trajectories for each attribute, as well as their averages over the different runs. Although the user will interact with the environment through a Web browser that runs on his local computer, simulation will occur on a back-end server that stores the models and their associated trajectories. We believe that this Web-based approach has key advantages over traditional schemes that require installing software on local computers, including ease of entry and the ability to share models. Moreover, as the environment's user community grows, it will let us move simulation into cloud services that provide scalable computing power and ensure rapid run times.

Visualizing the results of simulation runs will be another primary function of the modeling environment. Upon invoking the simulation mechanism, users will be able to specify the attributes, either aggregated or for individual agents, that they want to inspect. Once simulation has run to completion, the environment will display the predicted trajectory for each attribute in a separate graph, along with observations for the same attribute, if any are available. The environment will also report standard measures of fit, such as r^2 and squared error. The ability to compare simulated to observed trajectories is essential to evaluating models of social behavior, so we will also explore other ways to let users visualize and analyze model fits to data.

Searching the Space of Models

Computational aids for specifying, simulating, and evaluating formal models will undoubtedly aid researchers in their efforts to understand complex social behavior, but they downplay a key aspect of the scientific enterprise. This concerns the fact that there are many possible models and that scientists must *search* for candidates that explain observations in ways that are consistent with their theoretical commitments. We propose to develop computational methods that carry out search through such a model space and to embed them in modeling environment we will develop. Here we review our previous work on this topic and describe our plans to adapt it to our unified framework.

A REVIEW OF INDUCTIVE PROCESS MODELING

In previous research, Bridewell, Langley, Todorovski, and Dzeroski (2008) have described *inductive process modeling*, an approach to the automated construction of quantitative process models. The latter constitute a special case of the modeling formalism described earlier, comprising a set of entities with associated attributes and a set of processes, each containing a set of algebraic or differential equations. Given time-series observations for a set of entity attributes, along with background knowledge about generic types of entities and processes that may be involved, the aim is to produce a set of concrete models that explain the data in terms of interacting processes, ranked by their degrees of fit.

We have developed a number of systems that address this task, all of which operate in four stages. First, they use knowledge about generic entities and processes, combined with the observations to be explained, to generate a set of instantiated processes that include specific entities and functional forms but that do not specify parameter values. Second, they combine these instantiated processes in different ways to generate a set of candidate model structures. Third, they carry out gradient descent (combined with random restarts) through the space of parameters for each candidate structure, guided by fit to observations, to estimate that model's parameters. Finally, they order the parameterized model structures by this error measure and return a ranked list to the user for inspection.

We have tested this approach to constructing explanatory models experimentally, with considerable success in terms of variance explained, on a variety of data sets. Most of our testbeds have involved ecological data and models (Asgharbeygi et al., 2006), but we have also applied the approach to biomedical phenomena (Langley et al., 2006). Although these results have been limited to sets of differential equations at the aggregate level, we believe that the approach can be adapted to operate over the unified modeling formalism proposed in earlier sections. We now turn to our plans for carrying out these extensions.

EXTENDING GENERIC ENTITIES AND PROCESSES

The first step in extending the framework of inductive process modeling involves broadening the generic elements that are used to generate candidate structures. These include the generic entities that correspond to agent types in our unified framework, each of which is characterized by a number of attributes. In our previous work, generic entities specified ranges on such attributes' values that served as input to parameter estimation. However, our agent-based models incorporate *distributions* for such attribute values in order to support a variety of agents with values sampled from them. As a result, the generic entities must specify ranges on the parameters of these distributions. For example, if we want instances of an agent type to vary along an attribute that reflects resistance to a disease, say with a uniform distribution, then the generic entity must specify a range for this distribution's lower bound and another range for its upper bound. If different types of agents may occur in different numbers, then we must include ranges for these parameters as well.

We must also extend the structure of generic entities to support models that involve social relations. This will mean augmenting the notation for attributes of generic entities so that they may take other entities (constrained by their types) as values. These may specify not only relations to other individuals, but also to groups in which the entity is a member. Generic entities will also need to specify attributes that can modulate these values, such as the strength of friendship with another agent. Again, the aim is to characterize the *structure* of agents that can comprise a model, as opposed to the values that instantiate them.

Finally, we must extend our representation for generic processes. One necessary change involves support for processes that incorporate discrete actions, such as altering an agent's focus of attention. However, this should be straightforward, as the syntax will be quite similar to that for algebraic equations. A more substantive extension concerns support for conditions on processes. The issue here is that, although users may be willing to specify the forms that conditions can take (e.g., an attribute value exceeding a threshold), they may not want to specify either the number of such conditions or the attributes to which they refer. Nevertheless, generic processes can place constraints on them, say by specifying a set of agent attributes that may appear in conditions or by placing lower and upper bounds on their number.

EXTENDING METHODS FOR MODEL SEARCH

Recall that inductive process modeling separates search through the space of process models into distinct stages: generating alternative model structures and estimating parameter values for each structure. Naturally, we must augment both of these mechanisms to support automated model construction in the unified framework. Once we have extended our notation for generic entities and processes, as just outlined, our earlier methods for generating model structures should generalize to agent-based and social network accounts.

However, dealing with rule-based aspects of the framework will be more challenging, in that this will require search over sets of candidate conditions. Our previous methods for generating model structures have been exhaustive because the space typically involved only a few thousand alternatives, but introducing conditions will increase this substantially and require adoption of heuristic search techniques. Here we plan to adapt methods from the literature on rule induction for classification tasks (Langley, 1995), which typically search from more general hypotheses to more specific ones, guided by fits to training cases.

Extensions to parameter estimation should be relatively straightforward, again building directly on the expanded notations for generic entities and processes. For example, for each model structure, the system would still carry out gradient descent through the space of parameters that appear in that structure's equations. But it must also estimate the upper and lower bounds on distributions that characterize agent variations or, if one uses other distributions, the parameters that specify them. We may also need algorithms that estimate the percentages of different agent types within the population. The system must also induce the thresholds in process conditions, a task that may not lend itself to gradient descent techniques. Finally, we must find ways to make these mechanisms scalable to models with many agents and processes.

Finally, we must adapt the evaluation metrics used to guide parameter estimation and to rank the generated models. In previous work, we have used squared error averaged over all observed dependent attributes, but the new environment will be concerned not with individual agent attributes but with their average over entire populations of agent types. Users may also indicate that they care more about fitting observations for some aggregated attributes than others. We will explore this and other variations on metrics for model evaluation.

Preliminary Studies

In preliminary work, we have implemented an initial version of the modeling environment that supports some aspects of the proposed formalism. Users can access the environment through the World Wide Web and create/edit models that include a set of typed agents with associated attributes and a set of typed processes that include algebraic and differential equations, along with optional arithmetic conditions. The initial software supports entities and processes similar to those in Tables 1 and 2, but it cannot yet handle entity subclasses, it cannot parse equations or conditions that involve discrete attributes, it cannot average over multiple runs, and it currently supports only primary attributes, not the secondary attributes needed to model social relations.

Our initial implementation also lets users simulate the model, for a given step size and number of time steps, using the iterative methods described earlier, drawing on the Runge-Kutta method (Butcher, 2003) for solving ordinary differential equations. As noted, these calculations take place on the Web server rather than the user's computer, with results from the former passed to the latter. The modeling environment then displays graphs for the aggregated trajectories of requested attributes for the user's inspection. Figure 1 presents a screen shot of the Web interface, including a portion of an infectious disease model and plots of

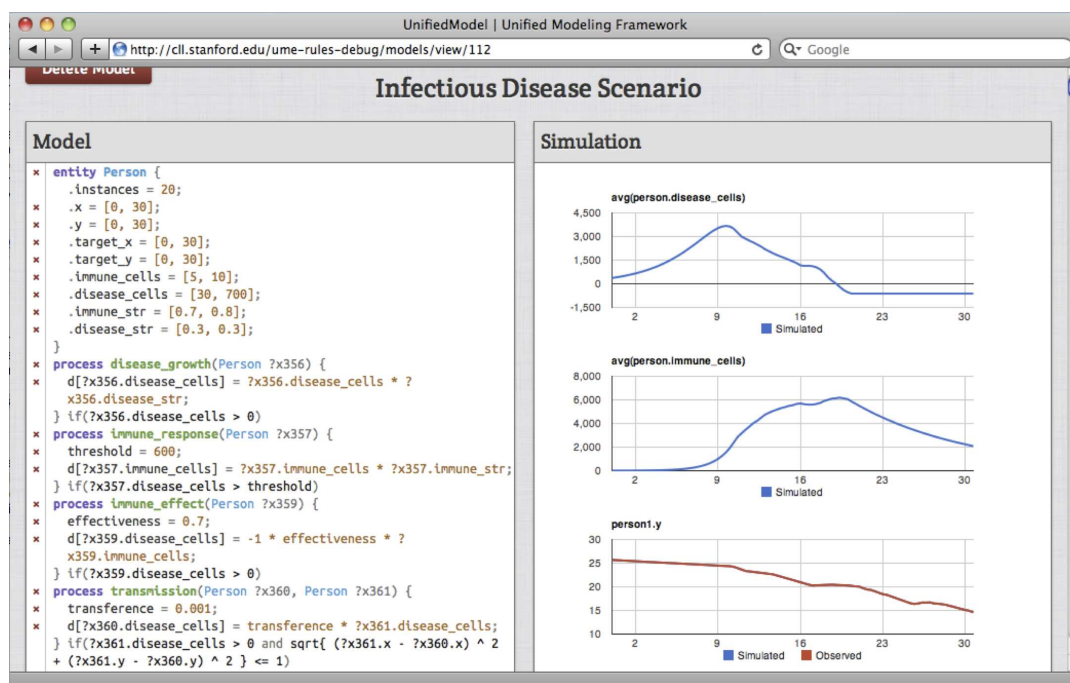


Figure 1. Preliminary graphical interface for the modeling environment. A portion of an infectious disease model appears on the left, while plots for aggregate attributes appear on the right.

average values obtained during a simulation run. Although our software remains preliminary, we have used it successfully in a course on modeling and simulation this semester at Arizona State University.

As explained earlier, we have also done considerable research on methods for automated construction of differential equation models that are organized into processes (e.g., Bridewell et al., 2008). These techniques should carry over directly to the new environment, but we must extend them to handle aspects of our framework that support agent-based models, social networks, and conditions on processes, as well as ways to scale them to models that involve many agents and grounded processes that relate them.

Research Schedule, Testbeds, and Personnel

The modeling framework that we have outlined has many facets, and implementing them in a usable and reliable software environment will take a multi-year effort. The results from our preliminary research have been encouraging, but they address only a fraction of the issues that we must tackle. However, we believe that four years will let us refine, implement, and test our ideas sufficiently to produce a modeling environment that many researchers will find attractive and effective.

Research during the project's first year will focus on finalizing the model formalism, consolidating our initial work on unifying agent-based, rule-based, and differential equation models, and testing the result on a number of modeling tasks. We will also refine the Web interface and make it available to a few external users. During the second year, we will extend the formalism and software to incorporate social relations, test the result on modeling tasks that involve them, and develop initial algorithms for automated induction of models that comprise agents and unconditioned processes. In the third year, we will extend model induction to handle conditions and social relations, test these techniques experimentally, and make our software available more widely. Efforts during the final year will focus on refining the interface in response to user feedback, ensuring our model induction methods are robust, and making the system's back end scalable to complex models.

Because it will be essential to test the environment's ability to support a variety of models, we will examine a number of domains during the project. Some candidates include:

- the spread and decline of infectious diseases, including the roles of disease resistance, immune response, social relationships, environmental factors, and health policies;

- the distribution of health information through individual interaction and social media (e.g., discussion groups) and its effects on disease prevention and treatment; and
- the interactions among population health, environmental pollutants, places of work and residence, and social relationships.

These testbeds are tentative, but each would appear to benefit from models that incorporate ideas from the four traditions we are aiming to unify. These areas are different enough that successful results will provide evidence for generality, yet similar enough that they should fit naturally into our framework.

In each case, we will not only develop models but attempt to secure data that will let us test both hand-crafted accounts and our methods for automated model construction. In addition, we will develop plausible models for each area, use them to generate synthetic data sets, and use these to test our mechanisms' ability to reconstruct model structures and associated parameters. We will also consider other testbeds that appear to provide useful challenges to our modeling environment. In addition, it will be important to build a user community that provides feedback from researchers who have developed models. This information will let us improve the system's functionality and interface, making it better suited to serve scientists' needs.

Our team includes two experienced researchers who have the knowledge and desire to carry the proposed work to completion. They will play distinct roles within the project appropriate to their expertise:

- Dr. Pat Langley will coordinate the project and supervise progress toward its goals, focusing especially on the unified modeling formalism and model induction. His distinguished record of research in computational models of cognition, biology, and ecology, including his work software environments for model development and discovery, make him the ideal person to manage the effort.
- Dr. Ted Selker will oversee the design, development, and evaluation of the Web-based interface. His extensive experience in interface design make him ideally suited for this role in the project.

In addition, a full-time postdoctoral researcher will implement and maintain the back-end software for storing, simulating, and constructing models, while an external consultant will extend the Web-based interface that lets users create and edit models, simulate them, and visualize the results.

The research project will also draw on three scientific advisors. Dr. Michael Barton, Professor of Anthropology at Arizona State University, has extensive experience in agent-based modeling and contributes to openabm.org, a Web site that supports work in this paradigm. Dr. Amar Das, Professor of Medicine at Stanford's Biomedical Informatics Research Center, is an expert in applying rule-based models to medical data and has a strong background in mathematical models for the social sciences. Dr. Vimla Patel, Senior Research Scientist at the New York Academy of Medicine, is an expert in both individual and group aspects of medical cognition and behavior. These advisors will provide feedback on the usefulness of the modeling environment, suggest promising tasks for testing, and provide high-level advice on model content.

Concluding Remarks

In summary, we propose to design, implement, and evaluate an innovative computational environment that supports modeling of complex systems in the social and life sciences. The environment will incorporate a formalism that combines ideas from agent-based models, differential equations, rule-based systems, and social networks, offering the abilities of each paradigm in a unified syntax. Moreover, users will construct models interactively in ways that minimize errors, and they will be able to visualize both model structures and simulation results. In addition, the environment will be accessible through a Web browser, thus lowering entry costs, supporting long-distance collaboration, and providing access to scalable cloud computing resources.

We will also develop computational methods that search the space of model structures and their associated parameters to help scientists uncover candidates that explain time-series data. We plan to demonstrate and evaluate our software on modeling tasks that involve interactions among social, health, and environmental factors. We have a strong technical team with expertise in scientific modeling, interaction design, and model discovery. We believe that the software we develop will revolutionize the way scientists construct, utilize, and evaluate models of complex systems, and that it will see wide adoption within the research community.

References

- Anderson, J. R., & Lebiere, C. (1998). *The atomic components of thought*. Mahwah, NJ: Lawrence Erlbaum.
- Asgharbeygi, N., Bay, S., Langley, P., & Arrigo, K. (2006). Inductive revision of quantitative process models. *Ecological Modelling*, 194, 70–79.
- Borgatti, S. P., Mehra, A., Brass, D. J., & Labianca, G. (2009). Network analysis in the social sciences. *Science*, 323, 892–895.
- Bradshaw, G. L., Langley, P., & Simon, H. A. (1983). Studying scientific discovery by computer simulation. *Science*, 222, 971–975.
- Bridewell, W., Langley, P., Todorovski, L., & Džeroski, S. (2008). Inductive process modeling. *Machine Learning*, 71, 1–32.
- Bridewell, W., Billman, D., Sánchez, J. N., & Langley, P. (2006). An interactive environment for the modeling and discovery of scientific knowledge. *International Journal of Human-Computer Studies*, 64, 1099–1114.
- Butcher, J. C. (2003). *Numerical methods for ordinary differential equations*. New York: John Wiley.
- Epstein, J. M., & Axtell, R. (1996). *Growing artificial societies: Social science from the bottom up*. Cambridge, MA: MIT Press.
- Forbus, K. D. (1984). Qualitative process theory. *Artificial Intelligence*, 24, 85–168.
- Forrester, J. W. (1961). *Industrial dynamics*. Waltham, MA: Pegasus Communications.
- Kauffman, S. A. (1993). *Origins of order: Self-organization and selection in evolution*. Oxford: Oxford University Press.
- Klahr, D., Langley, P., & Neches, R. (Eds.) (1987). *Production system models of learning and development*. Cambridge, MA: MIT Press.
- Laird, J. E. (2012). *The Soar cognitive architecture*. Cambridge, MA: MIT Press.
- Langley, P. (1995). *Elements of machine learning*. San Francisco: Morgan Kaufmann.
- Langley, P., Shiran, O., Shrager, J., Todorovski, L., & Pohorille, A. (2006). Constructing explanatory process models from biological data and knowledge. *Artificial Intelligence in Medicine*, 37, 191–201.
- Lee, B. Y., Bedford, V. L., Roberts, M. S., & Carley, K. M. (2008). Virtual epidemic in a virtual city: Simulating the spread of influenza in a US metropolitan area. *Translational Research*, 151, 275–87.
- Lotzmann, U., & Meyer, R. (2011). DRAMS – A declarative rule-based agent modelling system. *Proceedings of the 25th European Conference on Modelling and Simulation*. Krakow, Poland.
- Luke, S., Cioffi-Revilla, C., Panait, L., Sullivan, K., & Balan, G. (2005). MASON: A multi-agent simulation environment. *Simulation: Transactions of the Society for Modeling and Simulation International*, 82, 517–527.
- North, M. J., Collier, N. T., & Voss, J. R. (2006). Experiences creating three implementations of the RePast agent modeling toolkit. *ACM Transactions on Modeling and Computer Simulation*, 16, 1–25.
- Page, D., & Ong, I. (2006). Experimental design of time series data for learning from dynamic Bayesian networks. *Proceedings of the Pacific Symposium on Biocomputing*, 11, 267–278.
- Richmond, B., Peterson, S., & Vescuso, P. (1987). *An academic user's guide to STELLA*. Lyme, NH: High Performance Systems.
- Wilensky, U., & Rand, W. (in press). *An introduction to agent-based modeling: Modeling natural, social and engineered complex systems with NetLogo*. Cambridge, MA: MIT Press.